

TigerGraph Agentic Fraud Investigation HHGOA

Build an AI Agent for Fraud Investigation and Next-Best Action

Fraud teams at financial institutions are under constant pressure. Analysts must manually gather transaction history, trace money movement, identify connected accounts, review policies, assess risk, document findings, and decide what action to take. This process is slow, fragmented, difficult to scale, and often completes only after the money is already gone.

This hackathon challenges participants to build an Agentic Fraud Investigation Agent powered by TigerGraph that investigates fraud and recommends the next best actions when the available signals are uncertain.

Core Challenge / Design Guidelines

Build an AI agent that investigates fraud, creates and progresses cases, and recommends the next best action when available signals are uncertain.

The challenge is to accurately identify fraud patterns, determine what action to take, identify what additional evidence is needed, and decide when there is enough information to act.

The agent:

1. Investigate fraud when triggered by:
 - A fraud signal or risk score
 - A customer report
 - A fraud analyst
2. Gather and analyze evidence from:
 - Knowledge graphs
 - Transaction history
 - Device and identity signals
 - Account behavior
 - Prior fraud cases
 - External data sources
3. Assess the situation by
 - Identifying fraud patterns
 - Determine the likely type of fraud
 - Assessing the level of risk based on available evidence.
4. Create and progress a fraud case:
 - Create a case when investigation is warranted
 - Add new evidence and findings as the investigation progresses
 - Update the case status, risk assessment, and recommended actions
 - Maintain a record of decisions and actions taken

5. Use case memory to improve investigations:
 - Store relevant findings, decisions, actions, and outcomes from prior cases
 - Retrieve similar past cases when investigating new activity
 - Use prior case outcomes and analyst decisions to inform recommendations
 - Identify recurring fraud patterns, entities, and relationships across cases
 - Update memory as new cases are resolved
6. Gather additional evidence when needed through controlled, policy-approved actions, such as:
 - Asking an account owner to validate a transaction
 - Requesting step-up authentication
 - Requesting additional information from an analyst or approved party
7. Recommend or take one or more next actions based on the evidence, such as:
 - Allow or block a transaction
 - Block or monitor an account
 - Warn a customer
 - Create a fraud case
 - File a report
 - Request more evidence
 - Escalate to a fraud analyst
8. Operate within predefined policies and permissions:
 - The agent may recommend an action
 - Only authorized actions may be executed
 - Some actions may require human approval
9. Determine when to stop the investigation once enough evidence is available to take a defensible action.
10. Explain its reasoning, including:
 - What evidence was used
 - Why additional evidence was requested
 - Why the selected actions were recommended

The core investigation flow is:

1. Trigger - start an investigation based on fraud signal, customer report, analyst request or another event
2. Investigate - create or open a case and examine the relevant entities, transactions, relationships, behavior and prior cases
3. Gather evidence - collect the evidence needed to understand the activity and update the case as new information is found
4. Assess uncertainty - determine the level of risk, confidence in the assessment and whether enough evidence exists to act
5. Gather more evidence if needed - request or obtain additional information when uncertainty remains
6. Take one or more next actions - recommend or execute appropriate actions and progress the case
7. Explain the decision - clearly describe the evidence considered, remaining uncertainty and why the actions were selected
8. Update case memory - record the investigation, actions, decisions and outcomes so they can inform future cases

Actions such as sending customer messages, freezing accounts, blocking cards, refunding customers, updating CRM systems, or closing cases may be simulated, stubbed, or represented through mock APIs.

What Participants Build With

Required components

1. TigerGraph Savanna or Community Edition.
 - Use either for graph and vector storage and retrieval
 - Both are free to use for the hackathon
 - Savanna: <https://savanna.tgcloud.io>
 - Community Edition: <https://dl.tigergraph.com>
 - If you use Savanna, ensure auto-stop and auto-start are enabled
2. GSQL and graph algorithms
 - Use GSQL and TigerGraph graph algorithms for graph traversal, pattern detection, relationship analysis, and fraud investigation.
3. TigerGraph MCP
 - Use TigerGraph MCP to expose graph capabilities and data to the agent
 - <https://github.com/tigergraph/tigergraph-mcp>
4. GraphRAG
 - Use GraphRAG to ground the agent with relevant evidence and context
 - This may include connected evidence retrieved from the knowledge graph and information from documents such as fraud policies, procedures and typologies
 - Pass the relevant context to the LLM rather than simply passing raw data
5. User interface
 - Provide a user interface to demonstrate the investigation, case progression, evidence, uncertainty, recommendations, and next actions
 - This can be an analyst dashboard, conversational interface, case management view or another interface of your choice

Optional components

1. Agent framework
 - Participants may use the framework of their choice, such as LangChain, LangGraph, OpenAI Agents SDK, CrewAI, or another agent framework.
 - A custom agent implementation is also allowed.
2. LLM
 - Participants may use the LLM or combination of models of their choice.
 - The LLM should be used for reasoning, tool selection, evidence synthesis, and generating explanations rather than replacing graph analysis.
3. Additional tools and data sources
 - Participants may integrate external APIs, databases, fraud signals, identity systems, or simulated customer interactions.
 - External sources should complement the TigerGraph-based investigation.

Dataset

Here is the dataset:  **HHGOA_IIEE**

The dataset is built on the IEEE-CIS Fraud Detection data from Vesta Corporation, with every original row and column kept: about 590,000 card transactions over six months from about 13,500 customers, plus device and connection records for online transactions.

- Every transaction includes a risk score from the bank's fraud detection model. There is no "Is Fraud" flag.
- Closed investigations from the first four months are included, with cases that were confirmed as fraud and cases that were cleared.
- The dataset also includes the bank's fraud policy, five known fraud patterns, and relevant regulatory references.
- Twenty cases from the final two months are used as the benchmark. Every team will be evaluated on the same cases.
- You may extend the dataset with additional data or sources.
- Not every fraud pattern present in the data is documented.

Read the README inside the dataset first. It explains every file, every column, the answer format, and how the cases work.

Submissions

Submissions should include:

- Working agent
- GitHub repository
- Agent output on the 20 provided cases. For each case, one answer file contains
 - The case, including the internal investigation record, evidence, findings, decisions and actions taken.
 - The case should also be written to the graph
 - A suspicious activity report when required by policy
 - The next best action and required approval route recorded:
 - Before any additional evidence is requested
 - After any additional evidence is received
- 3–5 minute demo video showing the agent working end to end
- Technical blog post covering:
 - What you built
 - The architecture
 - How TigerGraph is used
 - The agentic capabilities you implemented
 - What you learned
 - What you would improve with more time
- Social media post on X or LinkedIn about what you built, your approach, or your experience building on TigerGraph, with a link to your blog post or demo. Tag @TigerGraphDB.

Judging Criteria

Investigation accuracy — 25%

Accuracy of identifying fraud patterns and quality of the fraud investigation and evidence gathered.

Next best action — 25%

Quality of the recommended actions, including how the agent handles uncertain or ambiguous fraud signals, determine when more evidence is needed and updates its recommendation as new evidence becomes available

Case summary and explainability — 10%

Quality of the case creation and progression, and clarity of the case summary, supporting evidence, reasoning, decisions and recommended actions

Agentic design and engineering — 15%

Quality of the agent architecture, tool use, workflow orchestration, memory, controls, permissions and overall implementation.

Innovation — 15%

Originality of the approach and use of graph, AI, GraphRAG, and agentic capabilities.

Demo quality and completeness — 10%

Clarity and effectiveness of the end-to-end demonstration.

What Success Looks Like

A successful solution should demonstrate that the agent can:

1. Investigate a fraud case from an initial trigger.
2. Gather relevant evidence from the graph and other available sources.
3. Identify fraud patterns and assess risk.
4. Create and progress a fraud case as new evidence and decisions are added
5. Recognize when the available evidence is not sufficient or uncertain
6. Gather additional evidence through controlled actions when needed.
7. Recommend one or more appropriate next actions and update those recommendations as new evidence becomes available
8. Explain the evidence, reasoning, uncertainty and decisions behind its recommendations.
9. Operate within defined policies, permissions and approval requirements
10. Use prior cases and outcomes as memory to inform new investigations
11. Present the investigation, case progression and recommended actions clearly through a usable interface.

The strongest solutions will show that the agent can move from uncertain fraud signals to a clear, defensible course of action, while maintaining a complete case record and using past investigations to improve future decisions

Support

TigerGraph Mentors and Judges answer questions on the TigerGraph Discord: <https://discord.gg/7JMkCAy9D3>

Or reach out directly to **Devanshu, DevRel at TigerGraph: +91 7404313376**

Join the TigerGraph WhatsApp group - quick doubts, announcements, and updates: [Click here](#)